Efficient Tree Draft for Long-Context Speculative Decoding

Brian J Chan
National Yang Ming Chiao Tung University
brian030128.cs14@nycu.edu.tw

Ning-Chi Huang
National Cheng Kung University
nchuang@gs.ncku.edu.tw

Kai-Chiang Wu
National Yang Ming Chiao Tung University
kcw@cs.nycu.edu.tw

Abstract

Speculative decoding is an effective technique for accelerating autoregressive models in memory-bound regimes. Prior work has explored tree-based drafting to speculate multiple candidates at each step, improving verification acceptance rates. However, existing systems fail to fully realize the potential of tree-based drafting, leaving significant performance overhead. In this paper, we present FastDraft, a plug-and-play module that improves the efficiency of tree-based drafting in long-context speculative decoding. FastDraft exploits both the shared-prefix I/O pattern across draft branches and the constrained tree structure of speculative decoding, enabling specialized kernels that reduce draft-phase overhead while preserving compatibility with existing inference frameworks. FastDraft requires no modifications to the target model, draft model, or sampling logic. We implement FastDraft within SGLang and compare it against SGLang's vanilla tree-based drafting backend, demonstrating substantially faster draft execution: up to 2.92× speedup in the draft phase and 1.86× higher throughput over SGLang under long-context settings.

1 Introduction

Autoregressive language models generate one token at a time, requiring the full KV cache to be loaded into memory for every generated token. This results in a significant memory I/O bottleneck, particularly in long-context settings. Speculative decoding Chen et al. [2023] has emerged as a promising solution that utilizes a lightweight draft model to propose several candidate tokens, which are then verified against the target model in a single forward pass thereby amortizing the cost of KV cache access. This verification step guarantees the alignment with the target distribution, preserving generation quality. However, tokens that are rejected during verification lead to wasted computation, making the efficiency of the draft phase critical.

To improve token acceptance rates, recent work Miao et al. [2024], Li et al. [2025b], Chen et al. [2025] extends speculative decoding from linear sequences to tree-based drafting, where multiple candidate tokens are proposed at each step. This forms a token tree that enables the verifier to evaluate multiple continuations simultaneously, increasing the expected number of accepted tokens per verification pass. However, existing serving systems do not exploit this sharing when computing draft attention. SGLang Zheng et al. [2024], for example, stores the KV cache of the shared context only once through RadixAttention, but its draft attention still processes every branch of the tree as an independent query and therefore re-reads the shared context from memory once per branch.

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

Because candidates share a long common context, this results in substantial redundant memory I/O that scales with tree width, undermining the efficiency gains of tree-based drafting.

A natural way to reduce draft-phase memory traffic is to reuse the KV cache across shared prefixes using prior prefix-aware attention kernels Juravsky et al. [2024], Pan et al. [2025], Yao et al. [2025], Ye et al. [2024], which group queries with overlapping prefixes and compute attention jointly. However, in tree-based speculative decoding, the binding constraint is not merely kernel-level efficiency, but also compatibility with CUDA Graph replay. Existing kernels are designed for general multi-level prefix hierarchies, and this generality requires dynamic input structures whose shapes, grouping relationships, and scheduling metadata change as the tree evolves. As a result, they sacrifice the static execution pattern needed to avoid repeated CPU-side planning and launch overheads in the lightweight draft phase. These kernels also waste compute at the suffix level. They process each branch's suffix as an independent request, but in a draft step a branch contributes a single query token, i.e., only R query rows per KV head under grouped-query attention with ratio R. Tensor-core attention kernels compute on query tiles of at least 16 rows, so each suffix tile is mostly padding: 12 of its 16 rows for all draft models we evaluate (Llama-3.2-1B and the Qwen3-4B EAGLE-3 head, R=4), and 8 of 16 for a Llama-3-70B-style configuration (R=8). Because the number of suffix tiles grows with batch size times tree width, this padded work grows with the serving load.

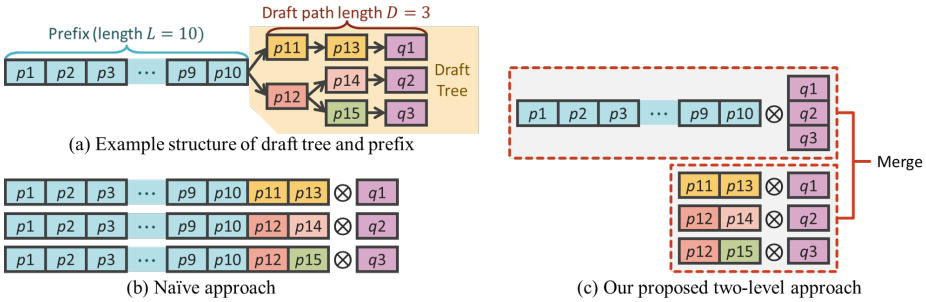
We observe that speculative draft trees have a much more constrained structure: a long verified prefix shared by all candidates, followed by a short branching suffix. Treating this two-level decomposition as a first-class primitive—and avoiding additional decomposition levels—makes the draft workload statically plannable, enabling a single scheduling plan per draft phase that can be reused across CUDA-Graph-replayed draft steps. Our cost-model analysis further shows that, for practical draft trees, additional intra-tree grouping levels do not amortize their merge and scheduling overheads. The same structure also removes the suffix padding: because all branches of a draft tree have the same depth, FastDraft packs the queries of $16/R$ branches into a single tensor-core tile and masks each query to its own branch, without breaking the static plan. FastDraft realizes this design through a specialized persistent attention kernel integrated into SGLang, combining shared-prefix KV reuse, padding-free suffix computation, and CUDA-Graph-compatible execution. In summary, this paper makes the following contributions:

- We reveal that existing tree-based speculative decoding frameworks fail to fully exploit the shared-prefix structure of draft trees during the draft phase, resulting in redundant KV-cache memory traffic and avoidable execution overhead. We further show that existing prefix-aware attention kernels do not close this gap: their general multi-level designs reduce some KV-cache traffic, but sacrifice the static planning and CUDA-Graph compatibility required for efficient lightweight draft execution, and they pad every branch's suffix queries to a full tensor-core tile, wasting most of the suffix computation.
- We make FastDraft compatible with CUDA Graphs by identifying and exploiting the static two-level structure of draft-tree attention. This enables GPU schedule planning to be performed once per draft phase and reused across draft steps, whereas existing approaches require repeated planning or CPU-side scheduling. The same structure lets FastDraft pack multiple branches into each tensor-core tile, eliminating suffix query padding.
- We integrate FastDraft into SGLang as a drop-in module and demonstrate substantial performance improvements on long-context speculative decoding workloads, achieving up to $2.92\times$ faster draft execution and up to $1.86\times$ higher throughput compared with the SGLang tree-based drafting baseline.

2 Methodology

We first observe the structure of a draft tree and its attention computation to generate draft tokens. Consider a draft tree $\mathcal{T}$ with N leaf queries $q_1, q_2, \ldots, q_N$. Each query q_i corresponds to a unique root-to-leaf path through $\mathcal{T}$, visiting a sequence of D internal nodes whose KV entries we denote $p_{i,1}, p_{i,2}, \ldots, p_{i,D}$. Following the PagedAttention framework Kwon et al. [2023] with page size being 1, we treat each node's KV entry as a single *page*, and use p throughout to emphasize this page-level view.

Figure 1(a) shows an example draft tree with three leaf queries, q_1, q_2, and q_3. The labels $p_{10}, p_{11}, \ldots, p_{15}$ are global page IDs identifying distinct KV entries in memory; sibling queries



(a) Example structure of draft tree and prefix

(b) Naïve approach

(c) Our proposed two-level approach

Figure 1: Queries and KV entries in draft phase of tree-based speculative decoding.

that share an ancestor reference the same page ID. For example, the tree encodes the following ancestor relationships: q_1 descends through $p_{10} \to p_{11} \to p_{13}$; q_2 through $p_{10} \to p_{12} \to p_{14}$; and q_3 through $p_{10} \to p_{12} \to p_{15}$. Here, the page p_{10} lies on the shared prefix common to all three queries, while p_{12} lies on a *shared sub-path* of the draft tree, used only by q_2 and q_3.

While the draft tree naturally exposes sharing across queries, existing draft attention implementations compute attention for each query independently, as in the naïve approach illustrated in Figure 1(b). This holds even when the KV cache itself is deduplicated: SGLang's RadixAttention Zheng et al. [2024] stores each shared page once, but every query still loads it during attention. Under the naïve approach, every query q_i attends over its full path, loading all KV pages along $p_1 \cdots p_{10}$, and $p_{i,1} \cdots p_{i,D}$ from memory. Although pages $p_1, p_2, \ldots, p_{10}$ are on the shared prefix, these KV pages are loaded repeatedly for each query, leading to substantial redundancy.

2.1 Two-Level Attention Computation

To reduce redundant KV loads during the draft phase, we adopt the shared-prefix decomposition of Hydragen Juravsky et al. [2024] and Cascade Inference Ye et al. [2024]; our contribution is its execution in the draft phase (Section 2.4). As illustrated in Figure 1(c), instead of computing the full attention over $\mathrm{KV}(q_i)$ for each query independently and loading its entire path of KV pages, we partition the KV cache into two parts, the shared prefix and the branching suffix. The shared prefix consists of the prompt and all previously accepted tokens, i.e., the full context that every branch of the draft tree attends to; its length L is the context length, not a property of the draft tree. A branching suffix is the shallow tree of candidate continuations generated during the draft phase. Throughout the paper, "prefix" always refers to this shared context, and segments of the draft tree that are shared by only a subset of branches are called shared sub-paths. The draft model attends over the shared prefix at every draft step. This holds for standalone draft models and for EAGLE-style drafters alike, since each draft token attends over the KV cache of the full context, which is exactly the traffic that FastDraft reduces. This induces a natural two-level structure:

$$\mathrm{KV}(q_i) = \underbrace{p_1, \ldots, p_L}_{\text{shared prefix}} \;\|\; \underbrace{p_{i,1}, \ldots, p_{i,D}}_{\text{branch-specific suffix}} .$$

The attention computation can be decomposed into three steps: a grouped prefix attention, per-query draft path attention, and a merge operation.

Step 1: Grouped prefix attention. First, we load the KV entries of the shared prefix (i.e., $K_{\mathrm{pfx}}, V_{\mathrm{pfx}}$) once and reuse them across all queries to compute attention scores. Because all N queries are batched, $K_{\mathrm{pfx}}, V_{\mathrm{pfx}}$ are loaded from HBM only once, the memory traffic can be reduced from $N \cdot L$ to L. For each query, a partial output and log-sum-exp accumulator are produced:

$$O_i^{(\mathrm{pfx})},\ \mathrm{lse}_i^{(\mathrm{pfx})} = \mathrm{Attn}\big(q_i,\ K_{\mathrm{pfx}},\ V_{\mathrm{pfx}}\big) \quad \text{for } i = 1, \ldots, N \tag{1}$$

Step 2: Per-query draft path attention. Second, the remaining attention over the branch-specific suffix is computed for each query independently. Each query q_i performs attention computation independently with its flattened draft path $K_{p_{i,1:D}}, V_{p_{i,1:D}}$.

$$O_i^{(\mathrm{sfx})},\ \mathrm{lse}_i^{(\mathrm{sfx})} = \mathrm{Attn}\big(q_i,\ K_{p_{i,1:D}},\ V_{p_{i,1:D}}\big) \quad \text{for } i = 1, \ldots, N \tag{2}$$

Step 2 is small in memory traffic but poorly suited to tensor-core execution. Each query q_i is a single draft token, which under grouped-query attention amounts to only R query rows per KV head, whereas tensor-core attention kernels process query tiles of at least 16 rows. Computing Step 2 per query, as prefix-sharing kernels do when they treat each suffix as an independent request, therefore leaves $16 - R$ of every 16 rows as padding: 75% of the tile for R=4, 50% for R=8, and 94% under multi-query attention (R=1). The number of such tiles is N per KV head and per batched request, so this padding grows with tree width and batch size. Section 2.4 shows how FastDraft removes it by packing the queries of several branches into one tile.

Step 3: Merge. We merge the partial outputs of the above two steps to produce the same mathematical output as full attention with standard online softmax Milakov and Gimelshein [2018]:

$$m_i = \max\big(\mathrm{lse}_i^{(\mathrm{pfx})},\ \mathrm{lse}_i^{(\mathrm{sfx})}\big) \tag{3}$$

$$O_i = \frac{e^{\mathrm{lse}_i^{(\mathrm{pfx})} - m_i} \cdot O_i^{(\mathrm{pfx})} + e^{\mathrm{lse}_i^{(\mathrm{sfx})} - m_i} \cdot O_i^{(\mathrm{sfx})}}{e^{\mathrm{lse}_i^{(\mathrm{pfx})} - m_i} + e^{\mathrm{lse}_i^{(\mathrm{sfx})} - m_i}} \tag{4}$$

for $i = 1, \ldots, N$, producing the final attention output for each query.

2.2 Analysis of Model Cost

We compare the I/O complexity of our two-level approach against naïve per-query full attention. Both approaches compute mathematically identical attention over the same queries and keys, so the arithmetic cost (FLOPs) is unchanged; only the data movement pattern differs. We therefore focus on HBM bandwidth overheads, specifically the data movement associated with KV-cache loading and intermediate writing. Besides, all costs are stated per KV head in units of one KV-token load. Under Grouped Query Attention (GQA) Ainslie et al. [2023] with ratio $R = H_q/H_{kv}$ and per-head dimension d, loading one KV entry (one pair of KV vectors) for each token costs $2d$ elements of HBM traffic, amortized over R query heads, and thus the unit cost per head is $c_{\mathrm{kv}} = 2d/R$.

Naïve Approach. Each of the N queries independently compute attention score with the KV entries of its full context, including the repeatedly loaded prefix whose length is L and the draft path whose length is D. Therefore, the memory I/O of a naïve approach is:

$$C_{\mathrm{naive}} = N \cdot (L + D)\,, \tag{5}$$

Our Two-Level Approach. The memory overhead of two-level approach is from all three steps mentioned in Section 2.1. Step 1 loads the KV cache of the shared prefix once and reuses it across all N queries. Step 2 loads each query's draft path independently. Both steps write their partial output (d elements) to the HBM. Step 3 a merge kernel reads the partial output and produce the final output. This comes with an additional cost γ for each query:

$$\gamma \;=\; \frac{4d+4}{2d/R} \;=\; R\big(2 + \tfrac{2}{d}\big) \;\approx\; 2R\,. \tag{6}$$

The extra HBM traffic per query per head is $4d + 4$ elements. Consider all three steps, and the total memory overhead of our two-level approach can be summarized as:

$$C_{\mathrm{2L}} = L + N \cdot D + \gamma N\,. \tag{7}$$

Savings. Memory overhead can be reduced by applying our two-level approach:

$$C_{\mathrm{naive}} - C_{\mathrm{2L}} = (N{-}1)\,L - \gamma N. \tag{8}$$

which is positive whenever $L > \gamma N/(N{-}1)$. For typical speculative decoding parameters (L in the hundreds to thousands, $N \le 16$, and $4 \le \gamma \le 20$), this condition is easily satisfied, and the savings scale linearly with the prefix length L.

2.3 Why Two Levels Suffice for Practical Draft Trees

For existing practical draft tree generation, adding intermediate grouping levels to the draft tree (i.e., batching queries that share a common sub-path) confers no additional I/O reduction beyond the two-level decomposition. This follows because each additional decomposition level introduces a merge overhead of at least γN (Eq. 6), while the most KV loads it can save is $(D-1)(N-1)$ in the best case that all queries share a common ancestor at the depth being $D-1$. An extra level is therefore beneficial only when

$$D \;>\; \frac{\gamma N}{N-1} + 1 \;\approx\; 2R + 1 \quad \text{for large } N\,.$$

This threshold should be interpreted as an optimistic I/O-only lower bound on the crossover depth. It assumes that the only cost of adding a grouping level is the HBM traffic needed to materialize and merge an additional partial attention result. In practice, an additional level also introduces fixed scheduling cost, extra kernel work, and reduced GPU utilization because the newly created subproblems are short and fragmented. To capture these effects, let λ denote the fixed per-level overhead converted into units of one KV-token load. Then an additional grouping level over a shared draft subpath of length $D-1$ is beneficial only if

$$(D-1)(N-1) > \gamma N + \lambda.$$

Equivalently, the crossover depth becomes

$$D > 1 + \frac{\gamma N + \lambda}{N-1}.$$

The equation suggests that adding grouping levels inside the draft tree is unlikely to be beneficial for practical speculative decoding workloads. Each additional level introduces fixed merging, scheduling, and kernel fragmentation overhead, while its only potential benefit is to reduce redundant KV traffic along shared sub-paths. This benefit is limited by the depth of the draft tree. In current systems, the branching suffix is short, so there are too few draft tokens over which intermediate prefix sharing can amortize the extra overhead.

Under the assumption that the current draft trees are short, the two-level decomposition is the preferred design point. It captures the dominant source of reuse, that is, the long verified prefix shared by all draft queries, while avoiding the additional costs introduced by finer-grained multi-level grouping. Additional levels may only become beneficial for substantially deeper draft trees, where shared sub-paths are long enough to offset their merge and scheduling overheads. We validate this assumption empirically in Section 3.1.

2.4 Two-Level Attention Kernel for Tree-Based Draft Decoding

Two-Level Persistent Kernel. FastDraft executes both levels of the draft attention, a long memory-bound shared prefix and k short compute-light suffixes, in a single persistent kernel followed by a lightweight merge of the partial results. At planning time, the work is divided into prefix chunks per request and KV head and suffix items per KV head, which are statically load-balanced across a fixed grid of thread blocks. For the prefix, the k draft queries of a request and the R query heads sharing each KV head form one query tile of kR rows, so every prefix KV tile is loaded from HBM once for all branches; the prefix is split along the KV dimension only as far as needed to occupy the GPU, and the number of thread blocks per SM is chosen at planning time. For the suffixes, a per-branch tile would carry only R useful query rows out of the 16-row tensor-core minimum, leaving the rest as padding. Since all branches have the same depth and their suffix KV is stored at a fixed stride, FastDraft instead packs $16/R$ branches of a request into one tile with a block-diagonal mask, recovering the active suffix length from the per-step metadata already patched during CUDA-Graph replay, so the plan stays static. Co-resident thread blocks run the suffix items alongside the memory-bound prefix items, largely hiding the suffix work.

Comparison with Prior Prefix-Sharing Kernels. Hydragen Juravsky et al. [2024] and FlashInfer's cascade attention Ye et al. [2024] also split attention into a shared prefix and per-sequence suffixes and merge the partial results through their log-sum-exp statistics, while FastTree Pan et al. [2025] and DeFT Yao et al. [2025] group queries along the internal topology of the tree. FastDraft differs in how the decomposition is executed for draft trees. First, these kernels process each level,

or each group of tree nodes, separately, with its own KV splitting, and combine the results in an additional merge; FastDraft schedules both levels and their KV chunks in one work queue and executes them in a single persistent kernel with a single reduction. Second, they treat each suffix as an independent request, so each suffix tile is padded to 16 query rows, whereas FastDraft packs several branches into each tile to remove this padding. Third, their scheduling metadata depends on shapes that change across draft steps, whereas FastDraft's plan is static and replayable under CUDA Graphs. As a result, FastDraft outperforms FlashInfer's cascade attention, FastTree, and DeFT in nearly all settings of the kernel benchmark (Section 3.2), and end to end, with the same accept length and verification time, it drafts 3.1–$4.0\times$ faster than Hydragen and 2.4–$3.3\times$ faster than DeFT (Table 1).

CUDA Graph Replay for Draft Loop. CUDA Graphs are important in the draft phase of speculative decoding because the draft model is small, so CPU launch and scheduling overhead can become a noticeable fraction of draft latency. FastDraft therefore treats CUDA Graph replay as a first-class part of the kernel design. Rather than merely capturing an individual attention call, FastDraft makes the repeated multi-step draft loop replayable without CPU-side scheduling in the critical path.

We build on the standard `plan()`–`run()` separation used by modern attention systems. The challenge is that, in a naïve implementation, `plan()` would still need to run before every draft step in order to rebuild the scheduling metadata for the current KV layout, causing CPU synchronization and disrupting the GPU execution flow. FastDraft avoids this by exploiting the shape invariance described in the next paragraph: the two-level draft-tree structure allows FastDraft to plan only once for a draft phase and reuse the same plan across subsequent draft steps.

During CUDA-Graph replay, FastDraft keeps the scheduling metadata produced by `plan()` fixed and updates only the per-step dynamic state on the GPU. The shared-prefix index buffers are constructed once and reused throughout replay, while the active suffix lengths and suffix KV indices are patched in preallocated maximum-depth buffers as the draft tree grows. Each captured step uses a stable workspace to avoid pointer aliasing inside the graph. As a result, the replayed draft loop consists only of GPU-side suffix-index patching, lightweight workspace updates, and persistent attention execution, with no repeated `plan()` call, host-side scheduler invocation, or host–device synchronization on the critical path.

Why Two Levels Enable Reuse. The key observation is that the two-level formulation produces the same planning structure in every draft step. In each step, exactly one shared-prefix part and exactly one suffix part are used for each branch. Thus, the mapping from the tree to the attention problem is fixed: branch i always maps to suffix part i. As the draft tree grows, the active suffix of each branch becomes longer, but the number of parts, their ordering, and their buffer layout remain unchanged. All planning metadata can therefore be allocated once for the maximum draft depth and reused across steps. The per-step dynamic state is limited to the active suffix lengths and the corresponding suffix KV indices, both of which are updated in preallocated buffers as the draft tree grows.

3 Evaluation

We conduct four sets of experiments to validate our approach: **(1)** a micro-benchmark comparing our proposed two-level decomposition against multi-level cascade attention to empirically confirm the optimality analysis mentioned in Section 2.3; **(2)** a comparison of existing shared-prefix attention kernels applied to the draft-tree attention problem; and **(3)** an end-to-end speculative decoding speedup analysis. **(4)** an ablation study of how each optimization contributes to the speed up.

3.1 Comparison Between Two-Level and Multi-Level Attention

We empirically verify the analysis of Section 2.3 by comparing the latency of two-level decomposition against the multi-level approach, where additional grouping levels are introduced within the draft tree at every branching point. All experiments use a prefix length of $L = 50$K, $H_q = 32$ query heads, $H_{kv} = 8$ KV heads (GQA ratio $R = 4$), and head dimension $d = 128$, matching the configuration of Llama-3-8B.

We sweep tree width $W \in \{4, \ldots, 16\}$ and depth $D \in \{2, \ldots 20\}$ under three draft-tree distributions that span the spectrum of sharing structure:

- **Min Sharing.** Each leaf follows a maximally distinct path through the tree, minimizing the KV cache shared among sibling queries within the draft tree. This is the least favourable scenario for multi-level grouping due to almost no shared sub-path to exploit.
- **Max Sharing.** All leaves share a single long draft path up to the penultimate level before diverging, maximizing the shared sub-path within the draft tree. This is the most favourable scenario for multi-level grouping, as the intermediate sharing benefit is the largest.
- **Tree Sampling (Realistic Drafting).** Draft trees are constructed by sampling from the speculative decoding process, accumulating log-probabilities along each path and retaining the top-k continuations at each expansion step to approximate the distribution of draft trees encountered in realistic speculative decoding.

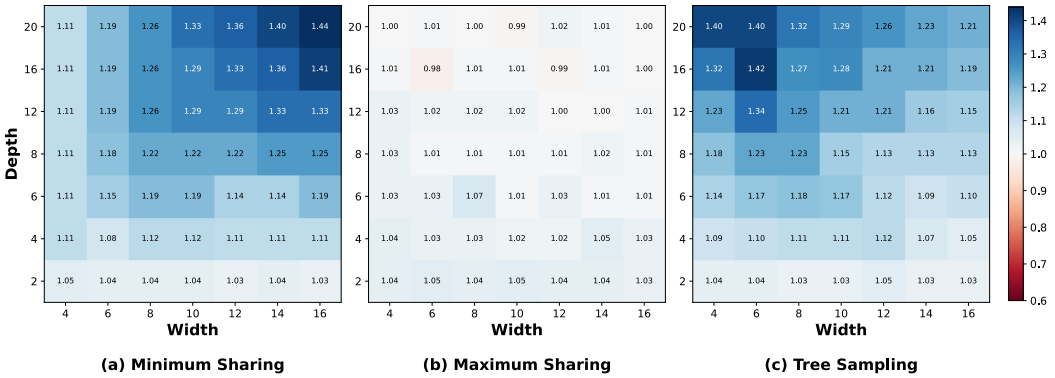


Figure 2: Two-level vs. multi-level attention speedup across tree width W and depth D ($L = 50$K, $H_q/H_{kv} = 32/8$, $d = 128$). Heatmaps of speedup ratio $T_{\text{multi}}/T_{\text{two-level}}$ (blue = two-level wins)

Figure 2 reports the speedup ratio $T_{\text{multi}}/T_{\text{two-level}}$ across all (W, D) configurations, where T_{multi} is the latency of our implementation of multi-level approach and $T_{two-level}$ is the latency of our two-level approach. A ratio greater than one indicates that the two-level scheme is faster. In the minimum sharing case, the two-level decomposition outperforms multi-level attention by $1.03\times$–$1.44\times$, with the advantage growing with tree depth as the per-level merge overhead accumulates in the multi-level scheme. In the maximum sharing case, which is the most favorable setting for multi-level grouping, multi-level attention exhibits only a very small crossover at large depths. Starting around $D = 16$, the best observed ratio drops to about $0.98\times$.

Under the tree sampling distribution, which reflects realistic drafting behavior, the two-level scheme remains faster across all tested configurations. This is the most relevant regime for speculative decoding systems, where draft depths are typically shallow. Speculative decoding methods with draft models from the same family (e.g., Llama-3.2-1B drafts for Llama-3.1-8B) usually perform best around $D = 6$–10; EAGLE-3 Li et al. [2025a] uses $D \leq 8$, SpecInfer Miao et al. [2024] uses $D = 8$. Even Sequoia Chen et al. [2025], which explores deeper trees, uses depths around $D = 18$ mainly in offloading settings where the tree must be deeper to offset communication and memory-transfer overheads.

Overall, these experiments support the assumption in Section 2.3 that practical draft trees are too shallow for additional sharing along sub-paths to provide a meaningful benefit. Thus, two-level decomposition is the preferred practical design point for tree-based speculative decoding.

3.2 Kernel Benchmark on Draft Tree Problem

In this section we evaluate the speedups brought by our kernel against existing prefix-IO-aware attention kernels on two-level draft tree structures. Here, three baselines are compared: FlashInfer's multi-level cascade attention Ye et al. [2025], FastTree Pan et al. [2025], and DeFT Yao et al. [2025]. Since these kernel implementations are optimized for H100 as stated in their paper, we conduct the experiments on H100. We evaluate prefix (context) lengths ranging from 2K to 32K tokens, batch size $\in \{1, 8\}$, and draft widths $k \in \{8, 16\}$. For each configuration, we sweep suffix lengths

from 1 to 16 and report the average over 50 runs for each tree structure, simulating the range of draft tree depths encountered in practice. All kernels are benchmarked in isolation with CUDA Graphs enabled, so that the results isolate attention computation from end-to-end framework overhead. Although some baseline kernels do not support CUDA Graph replay in fully dynamic serving workloads, this micro-benchmark uses a fixed tree structure within each set of 50 runs, making the input shapes, tree topology, and buffer layouts static and therefore CUDA-Graph-capturable for all evaluated kernels.

Figure 3 reports speedups over PagedAttention Kwon et al. [2023]. Our kernel consistently achieves the highest speedup across nearly all settings, with the advantage widening as prefix length and draft width grow. For example, at the setting of H_q=32, H_{kv}=8, B=8, k=16, our speedup rises from 2.20× at 2K to 4.92× at 32K, reflecting the increasing benefit of prefix-IO reuse as the shared KV cache dominates memory traffic. Among the baselines, FastTree performs best in the isolated kernel benchmark, but still trails our kernel by roughly 1.1–1.6× across settings, while FlashInfer frequently falls below 1×. Doubling the draft width from k=8 to k=16 consistently amplifies our lead by exposing more query-side parallelism per KV load, with peak speedups of up to 4.92×. This benchmark therefore isolates the kernel-only contribution to FastDraft's speedup.

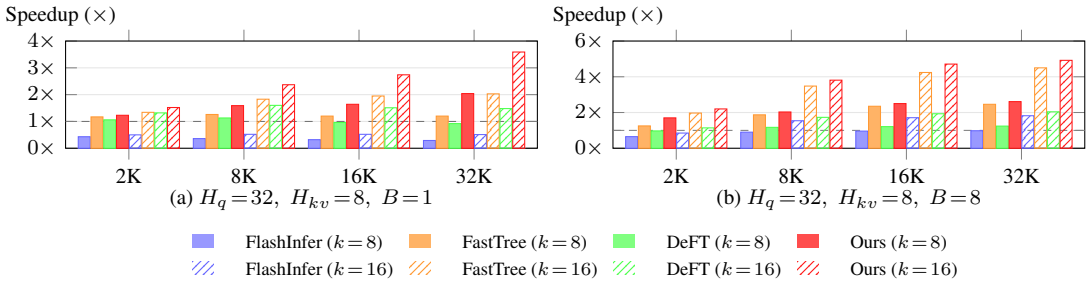


Figure 3: Speedup of tree attention kernels over paged attention across prefix lengths on H100 ($d=128$, FP16, suffix sweep 1-16 averaged). Solid bars denote $k=8$; hatched bars denote $k=16$. H_q and H_{kv} are the number of query and KV heads; B is the batch size.

3.3 End-to-End Speedup

Whereas Section 3.2 holds the tree shape fixed across each run set and thereby grants every kernel CUDA Graph capture, real serving exposes a draft loop in which baseline kernels fall back to ungraphed execution because their multi-level planning shape evolves across draft steps. The end-to-end results in this section therefore reflect both the kernel-level gap from Section 3.2 and the additional cost of ungraphed scheduling for baselines that cannot satisfy FastDraft's static-planning property.

We implement FastDraft as a toggleable module on SGLang v0.5.9 and benchmark output tokens per second alongside draft and verify latency for both classic speculative decoding and EAGLE3 Li et al. [2025b]. By classic speculative decoding we mean SGLang's `STANDALONE` mode: a separate, smaller model of the same family (Llama-3.2-1B for Llama-3.1-8B) with its own embedding and LM head drafts a token tree, which the target model then verifies. It differs from EAGLE only in that the draft model is a complete standalone model rather than a lightweight head on the target model's features; both draft trees, and both draft models attend over the full context KV cache. We also integrate the FastTree kernel into SGLang as the strongest prior tree-aware kernel baseline for the draft-tree attention problem, Hydragen Juravsky et al. [2024] as the original shared-prefix attention decomposition, and DeFT Yao et al. [2025] as a second tree-attention kernel. Like FastTree, Hydragen and DeFT plan around draft-step inputs whose shapes change at every step and therefore run without CUDA Graphs; the remaining baselines are compared at the kernel level only (Section 3.2). We evaluate across three long-context datasets, GovReport Huang et al. [2021], NarrativeQA Kočiský et al. [2017], and PG-19 Rae et al. [2019], covering context lengths ranging from 10K to 100K tokens. To demonstrate the generality of our method, we benchmark across a range of hardware devices, model sizes from 4B to 70B, and varying top-k settings. We also evaluate batched servings with batch sizes of 4 and 8 to test whether FastDraft's benefits persist beyond single-request decoding.

As shown in Table 1, FastDraft consistently delivers the best end-to-end performance in all evaluated settings, achieving the lowest draft latency and the highest speedup over autoregressive decoding, while leaving verification time and accept length unchanged relative to the SGLang baseline. These gains arise from treating CUDA-Graph compatibility as a first-class design constraint of the draft phase, rather than as a downstream engineering convenience. Hydragen and DeFT behave like FastTree: despite sharing prefix KV reads, they draft 2.4–4.0× slower than FastDraft and are slower than the SGLang baseline in every A6000 Llama setting. FastTree, on the other hand, although is a prefix-aware kernel that reduces KV-cache traffic, it is consistently slower than the non-prefix-aware SGLang baseline, because its reliance on dynamic input shapes precludes CUDA Graph capture of the draft loop, while FastDraft resolves this by exploiting the two-level draft structure, which uses static planning: a single plan is reused across draft steps under CUDA Graph replay, eliminating per-step host-side scheduling.

Table 2 shows that FastDraft's gains carry over to batched serving across different hardware settings. On NarrativeQA with a single RTX Pro 6000, FastDraft improves throughput over SGLang from 128.47 to 212.20 tokens/s at batch size 4 and from 131.42 to 226.16 tokens/s at batch size 8. On two A6000 GPUs, FastDraft improves throughput over SGLang from 103.97 to 193.18 tokens/s at batch size 4. Overall, FastDraft achieves up to 1.86× higher throughput over SGLang and up to 2.92× faster draft execution.

Table 1 reports decoding-time speedups, the phase FastDraft accelerates. Table 3 in the appendix reports full request latency including prefill, which takes 31–53% of the autoregressive request time here; the ratios are smaller, but FastDraft remains the fastest method in every setting.

3.4 Ablation Study

To quantify the contribution of each FastDraft component, we perform a step-by-step ablation starting from the CUDA-Graph-enabled SGLang draft backend. We first replace the SGLang draft backend with our specialized kernel, then enable CUDA Graph support for our kernel, and finally reuse the planning schedule across multiple draft steps. Figure 4 reports the draft-phase speedup on NarrativeQA at batch size 4, using Llama-3.2-1B as the draft model and Llama-3.1-8B as the target model on two A6000. Each component provides a consistent improvement, yielding speedups of 2.03×, 1.27×, and 1.13×, respectively. Combined, these optimizations achieve an overall draft-phase speedup of 2.92×.

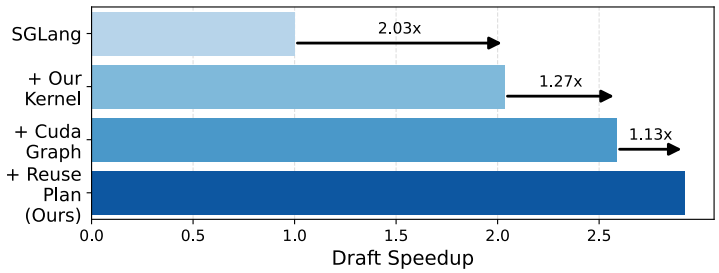


Figure 4: Draft-phase ablation study of FastDraft on NarrativeQA at batch size 4, using Llama-3.2-1B as the draft model and Llama-3.1-8B as the target model.

4 Conclusion

We presented FastDraft, a drop-in acceleration module for tree-based speculative decoding built on a single insight: the draft-phase attention workload is fundamentally two levels, a long verified prefix followed by a short branching suffix. We showed through a cost model and empirical evaluation that two levels are the preferred design point for practical draft trees, and designed a kernel that shares prefix reads, removes suffix query padding, and retains full CUDA Graph compatibility. Integrated into SGLang, FastDraft delivers up to 2.92× faster draft execution and meaningful end-to-end throughput gains, with consistent improvements across hardware (A6000 to H100), model sizes (4B to 70B), and long-context workloads from 10K to 100K tokens.

Table 1: Speedup is decoding-time speedup relative to autoregressive (AR) baseline. Bold indicates the shortest draft time and the highest speedup within each group. Values reported as mean ± std. Hydragen and DeFT run without CUDA Graphs (their draft-step inputs change shape every step) and are evaluated in the A6000 Llama settings.

Dataset	Target Model	Draft Model	Top-k	Method	Draft (s)	Verify (s)	Accept Len	Speedup vs AR
				A6000 × 2				
Gov Report	Llama-3.1-8B	Llama-3.2-1B	10	Ours	$\mathbf{1.75 \pm 0.33}$	1.38 ± 0.25	6.47 ± 0.97	$\mathbf{2.14 \pm 0.34}\times$
				SGLang	2.75 ± 0.67	1.41 ± 0.30	6.45 ± 1.04	$1.66 \pm 0.28\times$
				FastTree	4.63 ± 0.90	1.39 ± 0.27	6.45 ± 1.00	$1.17 \pm 0.19\times$
				Hydragen	6.48 ± 1.47	1.40 ± 0.28	6.50 ± 0.98	$0.91 \pm 0.16\times$
				DeFT	4.91 ± 1.13	1.41 ± 0.29	6.48 ± 1.01	$1.12 \pm 0.20\times$
NarrativeQA	Llama-3.1-8B	Llama-3.2-1B	10	Ours	$\mathbf{2.02 \pm 0.37}$	1.57 ± 0.28	6.56 ± 0.81	$\mathbf{2.17 \pm 0.27}\times$
				SGLang	4.07 ± 1.04	1.57 ± 0.26	6.52 ± 0.80	$1.44 \pm 0.20\times$
				FastTree	4.50 ± 0.77	1.55 ± 0.28	6.51 ± 0.82	$1.35 \pm 0.20\times$
				Hydragen	6.20 ± 1.39	1.57 ± 0.29	6.56 ± 0.80	$1.06 \pm 0.19\times$
				DeFT	4.94 ± 0.98	1.58 ± 0.26	6.56 ± 0.79	$1.25 \pm 0.20\times$
PG-19	Llama-3.1-8B	Llama-3.2-1B	10	Ours	$\mathbf{1.96 \pm 0.39}$	1.53 ± 0.29	6.58 ± 0.63	$\mathbf{2.20 \pm 0.21}\times$
				SGLang	3.82 ± 1.34	1.52 ± 0.28	6.65 ± 0.52	$1.49 \pm 0.25\times$
				FastTree	4.52 ± 0.62	1.51 ± 0.29	6.61 ± 0.54	$1.33 \pm 0.14\times$
				Hydragen	7.82 ± 1.27	1.59 ± 0.28	6.63 ± 0.58	$0.87 \pm 0.14\times$
				DeFT	6.49 ± 1.36	1.63 ± 0.37	6.56 ± 0.75	$1.00 \pm 0.16\times$
Gov Report	Qwen3-4B	Qwen3-4B-EAGLE3	10	Ours	$\mathbf{0.47 \pm 0.07}$	2.02 ± 0.31	3.31 ± 0.46	$\mathbf{2.05 \pm 0.30}\times$
				SGLang	0.64 ± 0.11	1.99 ± 0.30	3.32 ± 0.47	$1.96 \pm 0.29\times$
				FastTree	1.52 ± 0.22	1.99 ± 0.29	3.32 ± 0.47	$1.49 \pm 0.22\times$
NarrativeQA	Qwen3-4B	Qwen3-4B-EAGLE3	10	Ours	$\mathbf{0.67 \pm 0.19}$	3.17 ± 0.89	2.70 ± 0.74	$\mathbf{1.66 \pm 0.49}\times$
				SGLang	1.10 ± 0.32	3.18 ± 0.87	2.67 ± 0.75	$1.51 \pm 0.65\times$
				FastTree	2.29 ± 0.61	3.15 ± 0.86	2.67 ± 0.75	$1.21 \pm 0.36\times$
PG-19	Qwen3-4B	Qwen3-4B-EAGLE3	10	Ours	$\mathbf{0.56 \pm 0.12}$	2.62 ± 0.59	3.11 ± 0.73	$\mathbf{1.96 \pm 0.50}\times$
				SGLang	0.90 ± 0.21	2.64 ± 0.54	3.04 ± 0.68	$1.78 \pm 0.41\times$
				FastTree	1.85 ± 0.36	2.63 ± 0.53	3.04 ± 0.68	$1.43 \pm 0.37\times$
				H100 × 1				
Gov Report	Llama-3.1-8B	Llama-3.2-1B	15	Ours	$\mathbf{0.90 \pm 0.11}$	0.83 ± 0.11	5.94 ± 0.59	$\mathbf{1.84 \pm 0.19}\times$
				SGLang	1.52 ± 0.37	0.83 ± 0.11	5.95 ± 0.58	$1.40 \pm 0.19\times$
				FastTree	2.79 ± 0.32	0.83 ± 0.11	5.95 ± 0.60	$0.93 \pm 0.10\times$
NarrativeQA	Llama-3.1-8B	Llama-3.2-1B	10	Ours	$\mathbf{1.24 \pm 0.23}$	1.04 ± 0.19	6.58 ± 0.77	$\mathbf{1.74 \pm 0.21}\times$
				SGLang	2.59 ± 0.64	1.04 ± 0.19	6.58 ± 0.76	$1.14 \pm 0.17\times$
				FastTree	3.13 ± 0.45	1.04 ± 0.18	6.59 ± 0.75	$1.00 \pm 0.12\times$
PG-19	Llama-3.1-8B	Llama-3.2-1B	8	Ours	$\mathbf{1.29 \pm 0.28}$	0.93 ± 0.21	7.23 ± 0.53	$\mathbf{1.78 \pm 0.18}\times$
				SGLang	2.23 ± 0.67	0.93 ± 0.20	7.23 ± 0.59	$1.29 \pm 0.21\times$
				FastTree	3.12 ± 0.27	0.92 ± 0.19	7.28 ± 0.43	$1.02 \pm 0.08\times$
				RTX Pro 6000 × 4				
NarrativeQA	Llama-3.3-70B	Llama-3.2-1B	15	Ours	$\mathbf{1.33 \pm 0.38}$	2.86 ± 0.82	8.87 ± 1.74	$\mathbf{3.62 \pm 0.75}\times$
				SGLang	1.93 ± 0.72	2.89 ± 0.98	8.86 ± 1.78	$3.17 \pm 0.68\times$

Table 2: Comparison of autoregressive (AR), SGLang, and our method on NarrativeQA (target: Llama-3.1-8B, draft: Llama-3.2-1B, top-k=10, steps=7). B denotes batch size; Tput denotes decoding throughput in tokens per second. Tput speedups are relative to AR; draft speedups are relative to SGLang (AR has no draft phase).

Method	B	Draft (s)	Verify (s)	Tput (tok/s)	Tput Sp.	Draft Sp.
			RTX Pro 6000 × 1			
AR		—	—	101.13	1.00×	—
SGLang	4	7.07	2.18	128.47	1.27×	1.00×
Ours		2.85	2.22	212.20	2.10×	2.49×
AR		—	—	108.15	1.00×	—
SGLang	8	7.38	2.33	131.42	1.22×	1.00×
Ours		2.94	2.33	226.16	2.09×	2.51×
			A6000 × 2			
AR		—	—	96.65	1.00×	—
SGLang	4	8.72	2.72	103.97	1.08×	1.00×
Ours		2.99	2.71	193.18	2.00×	2.92×

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL https://arxiv.org/abs/2305.13245.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL https://arxiv.org/abs/2302.01318.

Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding, 2025. URL https://arxiv.org/abs/2402.12374.

Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. Efficient attentions for long document summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1419–1436, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.112. URL https://aclanthology.org/2021.naacl-main.112.

Jordan Juravsky, Bradley Brown, Ryan Ehrlich, Daniel Y. Fu, Christopher Ré, and Azalia Mirhoseini. Hydragen: High-throughput llm inference with shared prefixes, 2024. URL https://arxiv.org/abs/2402.05099.

Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The narrativeqa reading comprehension challenge, 2017. URL https://arxiv.org/abs/1712.07040.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test, 2025a. URL https://arxiv.org/abs/2503.01840.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty, 2025b. URL https://arxiv.org/abs/2401.15077.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS 24, page 932949. ACM, April 2024. doi: 10.1145/3620666.3651335. URL http://dx.doi.org/10.1145/3620666.3651335.

Maxim Milakov and Natalia Gimelshein. Online normalizer calculation for softmax, 2018. URL https://arxiv.org/abs/1805.02867.

Zaifeng Pan, Yitong Ding, Yue Guan, Zheng Wang, Zhongkai Yu, Xulong Tang, Yida Wang, and Yufei Ding. Fasttree: Optimizing attention kernel and runtime for tree-structured llm inference. In *Proceedings of Machine Learning and Systems*, 2025.

Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint*, 2019. URL https://arxiv.org/abs/1911.05507.

Jinwei Yao, Kaiqi Chen, Kexun Zhang, Jiaxuan You, Binhang Yuan, Zeke Wang, and Tao Lin. Deft: Decoding with flash tree-attention for efficient tree-structured llm inference, 2025. URL https://arxiv.org/abs/2404.00242.

Zihao Ye, Ruihang Lai, Bo-Ru Lu, Chien-Yu Lin, Size Zheng, Lequn Chen, Tianqi Chen, and Luis Ceze. Cascade inference: Memory bandwidth efficient shared prefix batch decoding, February 2024. URL https://flashinfer.ai/2024/02/02/cascade-inference.html.

Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for llm inference serving, 2025. URL https://arxiv.org/abs/2501.01005.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: efficient execution of structured language model programs. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.

Limitations FastDraft is most effective in long-context settings where the draft phase contributes a substantial portion of the overall decoding latency. When the context is short, the shared-prefix region is small, so the opportunity to reduce redundant KV-cache loads is limited. This effect is also reflected quantitatively in our kernel benchmark Section 3.2: at a prefix length of 2K tokens, batch size 1, FastDraft achieves only around $1.1\times$ speedup over PagedAttention, compared to significantly larger gains at longer prefixes. In this regime, the benefit of our prefix sharing kernel becomes correspondingly smaller.

In addition, the end-to-end speedup depends on how much time is spent in the draft phase relative to verification. If the draft phase is already short compared to the verify phase, then reducing draft latency yields only limited overall improvement, even if the draft kernel itself is accelerated. Therefore, the gains of FastDraft are largest when draft-side attention is a meaningful bottleneck, and smaller when verification dominates the decoding step.

Latency including prefill. Table 3 complements Table 1 with speedups of the full request latency (prefill and decoding, 512 output tokens) over autoregressive decoding, for the A6000 settings. The same runs are used: decoding speedup is the ratio of mean decoding times as in Table 1, and full-latency speedup the ratio of mean request latencies (std over per-prompt ratios). FastDraft accelerates only decoding, so its full-latency gain shrinks as prefill takes a larger share of the request.

Table 3: Speedup over autoregressive decoding on A6000×2, decoding only vs. full request latency including prefill (512 output tokens). Prefill share is the fraction of the autoregressive request latency spent in prefill.

Target / Draft	Dataset	Prefill share	Ours	SGLang	FastTree
Llama-3.1-8B / Llama-3.2-1B	Gov Report	31%	$\mathbf{2.14}$ / $\mathbf{1.43 \pm 0.15}$	1.66 / 1.21 ± 0.14	1.17 / 1.00 ± 0.10
	NarrativeQA	51%	$\mathbf{2.17}$ / $\mathbf{1.17 \pm 0.14}$	1.44 / 1.03 ± 0.12	1.35 / 1.01 ± 0.07
	PG-19	51%	$\mathbf{2.20}$ / $\mathbf{1.18 \pm 0.22}$	1.49 / 1.01 ± 0.19	1.33 / 0.97 ± 0.06
Qwen3-4B / Qwen3-4B-EAGLE3	Gov Report	33%	$\mathbf{2.05}$ / $\mathbf{1.47 \pm 0.13}$	1.96 / 1.44 ± 0.13	1.49 / 1.25 ± 0.11
	NarrativeQA	53%	$\mathbf{1.66}$ / $\mathbf{1.19 \pm 0.14}$	1.51 / 1.15 ± 0.14	1.21 / 1.06 ± 0.13
	PG-19	53%	$\mathbf{1.96}$ / $\mathbf{1.25 \pm 0.14}$	1.77 / 1.21 ± 0.12	1.43 / 1.12 ± 0.10

Each cell: decoding speedup / full-latency speedup.

Reproducibility. The full implementation and scripts required to reproduce the main experiments are available at `https://github.com/brian030128/fast-draft/tree/master`. All experiments are conducted on multiple virtual machines provisioned through verda.com, a cloud provider. Each virtual machine is equipped with GPUs of a single type: NVIDIA RTX A6000, NVIDIA H100, or NVIDIA RTX PRO 6000. The machines use the x86_64 architecture with 60 online CPUs, AMD EPYC 9555 64-Core processors, 128 GB of RAM, and a 500 GB SSD. Running the full set of reported experiments takes approximately two full days.

Assets and licenses. Our experiments use existing datasets, model checkpoints, and software systems, which are credited through the corresponding citations. The datasets include GovReport Huang et al. [2021], licensed under CC BY 4.0; NarrativeQA [Kočiský et al., 2017], licensed under the Apache License 2.0; and PG-19 Rae et al. [2019], licensed under the Apache License 2.0. The Llama-family model checkpoints, including Llama-3.1-8B, Llama-3.2-1B, and Llama-3.3-70B, are used under the corresponding Llama 3.1, Llama 3.2, and Llama 3.3 Community Licenses and Acceptable Use Policies. The Qwen-family checkpoints, including Qwen3-4B and the Qwen EAGLE draft checkpoints used in our experiments, are used under the Apache License 2.0. The software assets include SGLang Zheng et al. [2024], FlashInfer Ye et al. [2025], FastTree [Pan et al., 2025], EAGLE3 Li et al. [2025a], and DeFT Yao et al. [2025]. SGLang, FlashInfer, FastTree, and EAGLE are licensed under the Apache License 2.0, while DeFT is licensed under the MIT License. We use these assets only for benchmarking and comparison, and comply with their licenses and terms of use. The released FastDraft code will include license and attribution information for all reused assets.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes].

   Justification: The abstract and introduction state the paper's main contributions and scope: FastDraft is a drop-in acceleration module for tree-based speculative decoding in long-context settings, built around a two-level shared-prefix/suffix attention formulation, specialized kernel execution, and CUDA-graph-compatible draft execution. The quantitative claims in the abstract, including up to $2.66\times$ draft-phase speedup and $1.77\times$ throughput improvement, are supported by the evaluation. The claims are also scoped to the draft phase and end-to-end long-context serving, and do not claim changes to the target model, draft model, sampling logic, or output distribution.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes].

   Justification: The paper includes a separate Limitations section. It discusses that FastDraft is most effective in long-context settings where the shared prefix is large and draft-side attention is a meaningful bottleneck. The paper also notes that gains are smaller for short contexts, where there is less redundant KV-cache traffic to remove, and when verification dominates total decoding latency.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes].

   Justification: The paper includes an analytical memory-traffic cost model for the two-level decomposition. The assumptions of this analysis are stated in the methodology section, including the paged-attention view, per-KV-head accounting, GQA ratio, head dimension, prefix length, suffix depth, number of branches, and merge overhead. The resulting formulas for naive attention, two-level attention, and the crossover condition for additional decomposition levels are derived in the main text and then empirically validated in the evaluation. No additional unproved theoretical claims are used beyond this cost-model analysis.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes].

   Justification: The paper discloses the information needed to reproduce the main experimental results independent of code release, including the evaluated datasets, model pairs, hardware platforms, draft top-k settings, batch sizes, prefix-length ranges, GQA configurations, suffix-length sweeps, and baselines. The paper describes the benchmark methodology for both the kernel-level experiments and end-to-end serving experiments, and reports the relevant metrics, including draft latency, verification latency, throughput, and speedup, with mean and standard deviation where applicable. These details provide a reasonable path to reproducing the experiments; the implementation and scripts will additionally be provided as review materials.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation,

it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes].

   Justification: The supplemental material provides the implementation and scripts needed to reproduce the main experimental results. The experiments use publicly available datasets and model checkpoints, and the paper describes the evaluated datasets, model pairs, hardware settings, and benchmark configurations. The supplemental instructions specify the environment setup, data preparation, and commands for running the kernel benchmarks and end-to-end serving benchmarks for FastDraft and the baselines.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: The paper does not involve model training, so optimizer choice and training hyperparameters are not applicable.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: The end-to-end experimental results report mean and standard deviation for draft latency, verification latency, and speedup across benchmark runs/examples. The tables explicitly indicate that values are reported as mean $\pm$ standard deviation. For kernel-level microbenchmarks, the paper reports averaged latency/speedup over controlled configuration sweeps, where the measurements are intended to isolate deterministic kernel performance rather than model-quality variability. These statistics cover the experiments supporting the main performance claims.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes].

Justification: The paper reports the GPU configurations used for the experiments, including H100, A6000, and RTX Pro 6000 devices, as well as the model pairs, batch sizes, top-k settings, and benchmark configurations.The reported experiments required approximately two days of time in total. We did not use substantially more compute beyond the reported experiments, aside from routine debugging and preliminary validation runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes].

   Justification: We have reviewed the NeurIPS Code of Ethics and believe the research conforms to it. The paper presents a systems optimization for tree-based speculative decoding and evaluates performance on public datasets and model families; it does not involve human-subject experiments, collection of private or sensitive data, or deployment in high-stakes decision-making settings. We have also preserved submission anonymity and reported reproducibility and compute-resource information in the checklist.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes].

    Justification: This work is a systems optimization for long-context LLM inference, so its societal impacts are mostly indirect. Positively, faster speculative decoding can reduce latency, serving cost, making long-context LLM applications more accessible. Negatively, cheaper and faster inference may also lower the cost of misuse of existing generative models, including spam, disinformation, impersonation, or other harmful automated content generation.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: [N/A]

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes].

    Justification: The paper includes an "Assets and licenses" paragraph that credits the public datasets, model checkpoints, and software systems used in the experiments, including GovReport, NarrativeQA, PG-19, Llama-family and Qwen-family models, SGLang, FlashInfer, FastTree, and DeFT. The paragraph states that these assets are used under their respective licenses and terms of use, and that the released code will include license and attribution information for reused assets.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: [N/A]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.